

Rubik's Cube Solver — Comprehensive Documentation

An interactive, 3D web-based Rubik's Cube Solver that enables users to scan their physical cube with a webcam, verify the colors, solve it using advanced algorithms, and view step-by-step solution animation in real-time.

Core Features & Functionality

1. Webcam-Based Cube Scanning

- **Real-time video capture** from the user's device camera
- **Live mirror mode** for comfortable aiming (with automatic correction on capture)
- **Sequential face scanning** - guides users through all 6 faces in the correct order (F, R, B, L, U, D)
- **Visual feedback** - displays current face indicator with color dots and 3D cube orientation preview
- **Overlay graphics** - canvas-based visual guides for proper alignment

2. Color Detection & Classification

- **RGB to HSV conversion** - robust color space analysis for consistent detection
- **Intelligent color classification** - uses hue, saturation, and value thresholds:
 - **White (U)**: Low saturation, high brightness
 - **Yellow (D)**: 35-65° hue range, moderate-high brightness
 - **Red (R)**: 0° or 340°+ hue, high saturation
 - **Orange (L)**: 10-35° hue range, high saturation
 - **Green (F)**: 80-165° hue range, moderate saturation
 - **Blue (B)**: 180-270° hue range, high saturation
- **Adaptive thresholds** - accounts for lighting variations and camera differences

3. Interactive Verification Screen

- **2D Cube Net Representation** - displays all 6 faces in their proper spatial arrangement

- **Color Editing** - click any sticker to repaint it manually
- **Color Palette** - 6-color selector for quick corrections
- **Utility Functions:**
 - Clear all colors
 - Flip faces (mirror correction)
 - Batch color selection
- **Real-time validation** - instant feedback on cube legality

4. Cube Validation Engine

Comprehensive physical legality checks before solving:

- **Duplicate Detection** - ensures each face has unique piece identifiers
- **Corner Orientation Validation** - verifies corner twist parity (sum mod 3 = 0)
- **Edge Flip Validation** - verifies edge flip parity (sum mod 2 = 0)
- **Permutation Parity** - checks that corner and edge permutation parities match
- **Detailed Error Messages** - guides users to fix specific problems

5. Advanced Solving Algorithm (Kociemba)

- **Optimal/Near-optimal Solutions** - typically solves any valid cube in 20 moves or less
- **Asynchronous Processing** - runs in Web Worker to prevent UI freezing
- **Progress Feedback** - simulated progress updates during initialization (10-90%) and solving
- **Error Handling** - graceful failure with meaningful error messages
- **Support for All Orientations** - correctly handles any starting orientation

6. 3D Visualization & Animation

- **Three.js Rendering** - high-performance WebGL 3D graphics
- **Realistic Cube Model:**
 - Individually colored stickers (facelets) on each cube piece
 - Rounded box geometry for modern aesthetics
 - Proper internal black material
 - Correct piece hierarchy and transformations
- **Smooth Animations** - easeInOutCubic interpolation for natural motion
- **Interactive Controls:**

- Orbit Camera - rotate view with mouse/touch
- Zoom support
- Auto-orientation reset

7. Step-by-Step Solution Playback

- **Move Notation Display** - standard Rubik's cube move notation (R, U, F, B, L, D, M, E, S, etc.)
- **Move Descriptions** - human-readable explanation of each move
- **Playback Controls:**
 - First/Previous/Next/Last move navigation
 - Auto-play animation with adjustable speed (0.5x to 2.0x)
 - Move counter showing progress (e.g., "Move 7 of 15")
- **Speed Control** - granular slider for animation timing
- **Pause/Resume** - smooth pause during auto-play

8. State Management

- **Session Persistence** - maintains cube state across phases
 - **Multi-phase Workflow:**
 - Phase 1: Scan
 - Phase 2: Verify
 - Phase 3: Solve
 - Phase 4: Animate
 - **Step Indicator Progress** - visual feedback with completed steps, active step, and upcoming steps
-



Technical Architecture

Application Structure

```
App (Main Controller)
├─ CubeState (State Management)
├─ CubeScanner (Webcam & Color Detection)
├─ CubeSolver (Algorithm Bridge)
├─ Cube3D (3D Visualization)
├─ SolutionAnimator (Playback Control)
└─ DOM Controllers (UI Binding)
```

Data Flow

```
Webcam Capture
  ↓
Color Extraction
  ↓
Color Classification
  ↓
CubeState Update
  ↓
Preview Rendering
  ↓
Manual Verification
  ↓
Validation Check
  ↓
Web Worker Solve Request
  ↓
Solution Generation
  ↓
3D Animation & Playback
```

Tech Stack

Frontend Frameworks & Libraries

Category	Technology	Version/Source	Purpose
----------	------------	----------------	---------

3D Graphics	Three.js	v0.170.0 (CDN)	WebGL rendering, 3D cube visualization
Solver Algorithm	CubeJS	Latest (CDN)	Kociemba algorithm implementation
Module System	ES6 Modules	Native	Code organization and imports

Core Technologies

- **HTML5** - Semantic structure with canvas for scanning overlay
- **CSS3** - Modern features:
 - CSS custom properties (variables)
 - Flexbox & Grid layouts
 - Backdrop filters (glassmorphism)
 - Gradients and animations
 - Media queries for responsiveness
- **JavaScript** - ES6+ with:
 - Class-based OOP architecture
 - Promise-based async operations
 - Template literals
 - Destructuring and spread operators

Specialized JavaScript APIs

- **getUserMedia() API** - Camera access and video stream management
- **Canvas API** - Image data extraction, overlay graphics
- **Web Workers** - Asynchronous heavy computation
- **requestAnimationFrame()** - Smooth animations
- **Promise/Async-Await** - Asynchronous flow control

Libraries & Dependencies

Three.js Ecosystem

```
import * as THREE from 'three';
import { OrbitControls } from 'three/addons/controls/OrbitControls.js';
import { RoundedBoxGeometry } from
'three/addons/geometries/RoundedBoxGeometry.js';
```

Why These?

- **THREE**: Industry-standard WebGL abstraction
- **OrbitControls**: User-friendly 3D camera manipulation
- **RoundedBoxGeometry**: Modern cube aesthetics without hard edges

CubeJS Library

```
importScripts('https://cdn.jsdelivr.net/npm/cubejs/lib/cube.js');
importScripts('https://cdn.jsdelivr.net/npm/cubejs/lib/solve.js');
```

Functions Used:

- `Cube.initSolver()` - Loads Kociemba lookup tables
- `Cube.fromString(faceletString)` - Constructs cube from facelet notation
- `cube.solve()` - Generates optimal/near-optimal solution
- `cube.cp` - Corner piece positions
- `cube.ep` - Edge piece positions
- `cube.co` - Corner orientations
- `cube.eo` - Edge orientations



Web Worker Logic (solver.worker.js)

Purpose

Offloads CPU-intensive cube solving operations to prevent UI blocking, ensuring smooth user experience during solving.

Worker Lifecycle

1. Initialization Phase

```
action: 'init'
↓
Cube.initSolver() // Loads 1.4MB lookup tables
↓
action: 'init_done' (on success)
action: 'init_error' (on failure)
```

Performance Note: First initialization takes ~2-3 seconds. Cached after first use.

2. Solving Phase

```
action: 'solve'
payload: { faceletString: 'UF...BD' } // 54-character face encoding
↓
validateCube(cube) // Physical legality checks
↓
cube.solve() // Runs Kociemba algorithm
↓
action: 'solve_done' with moves array
```

Validation Engine

Multi-layer Validation:

1. Piece Uniqueness

```
const cPerm = new Set(cube.cp);
if (cPerm.size !== 8) // Invalid corners
```

2. Corner Twist Parity

```
let cTwist = cube.co.reduce((a,b) => a+b, 0);
if (cTwist % 3 !== 0) // Invalid corner rotation
```

3. Edge Flip Parity

```
let eFlip = cube.eo.reduce((a,b) => a+b, 0);  
if (eFlip % 2 !== 0) // Invalid edge orientation
```

4. Permutation Parity

```
// Count inversions in corner and edge permutations  
if (cornerInversions % 2 !== edgeInversions % 2) // Parity mismatch
```

Message Protocol

```
// Main → Worker  
{ action: 'init' | 'solve', payload: {...} }  
  
// Worker → Main  
{  
  action: 'init_done' | 'init_error' | 'solve_done',  
  success: boolean,  
  moves: Array<string>,  
  error: string  
}
```

Error Handling

- **Graceful Failures** - worker sends error messages without crashing
- **Detailed Diagnostics** - error messages identify specific invalid piece types
- **Fallback Messages** - user-friendly language in main app layer

User Experience Features

Professional UI Design

- **Light Blue-Slate Theme** - clean, modern aesthetic
- **Glassmorphic Cards** - backdrop blur and semi-transparent elements
- **Smooth Transitions** - 150ms-400ms CSS animations
- **Responsive Layout** - mobile-optimized (breakpoints: 768px, 480px)

- **Accessibility** - high contrast text, focus indicators

Feedback Systems

- **Progress Indicators** - step indicators with completion checkmarks
- **Validation Messages** - color-coded success/error feedback
- **Move Counter** - real-time progress during animation
- **Loading States** - progress bar during solving initialization
- **Visual Cues** - hover effects, active states, disabled states

Typography System (Mixed Professional)

- **Playfair Display** - premium serif for main titles
 - **Outfit** - geometric sans for section headings
 - **Inter** - clean body text and descriptions
 - **Poppins** - rounded sans for UI elements and buttons
 - **IBM Plex Mono** - monospace for technical notation
-



Performance Optimizations

Browser Rendering

- **CSS Transforms** - hardware-accelerated animations
- **requestAnimationFrame** - synchronized with monitor refresh rate
- **Debounced Events** - prevents excessive re-renders
- **CSS Variables** - centralized theme management (1 pass to change theme)

Computation

- **Web Worker Threading** - solver runs on separate thread
- **Progressive Loading** - displays UI while Three.js loads
- **Lazy Initialization** - Kociemba lookup tables load only when needed
- **Canvas Caching** - reuses overlay canvas between frames

Memory Management

- **Texture Atlas** - single texture for all cube stickers
- **Geometry Reuse** - shared RoundedBoxGeometry instances

- **Event Delegation** - single listener for multiple stickers
 - **Proper Cleanup** - scene disposal after use
-

Workflow Phases Explained

Phase 1: Scan

Goal: Capture all 6 faces via webcam

- User guides each face into view
- System detects dominant colors in center 3×3 grid
- Visual feedback shows progress
- Can skip to manual entry if camera unavailable

Phase 2: Verify

Goal: Correct any scanning errors

- 2D cube net shows all scanned colors
- User clicks stickers to manually edit
- System validates configuration in real-time
- Shows error if invalid (helps user fix it)

Phase 3: Solve

Goal: Generate solution sequence

- Displays solving progress
- Worker processes cube in background
- Shows solution length when complete
- Transitions to animation phase

Phase 4: Play

Goal: Visualize solution step-by-step

- 3D cube renders with OrbitControls
- User navigates through moves or auto-plays
- Move descriptions explain each step
- Speed slider controls animation timing



Data Security

- **Local Processing** - all computation happens client-side
 - **No Server Required** - static web app, no backend
 - **No Data Collection** - no analytics, no tracking
 - **Camera Access** - user must grant explicit permission
 - **HTTPS Recommended** - for secure camera access (localhost exempt)
-



Browser Compatibility

Required Support

- ES6 Modules
- getUserMedia() API
- Canvas 2D Context
- WebGL/Three.js

Tested On

- Chrome 90+
- Firefox 88+
- Safari 14+
- Edge 90+

Not Supported

- IE 11 (no ES6 modules)
 - Older mobile browsers (camera permission handling differs)
-



Cube State Encoding

Facelet Notation (54 characters)

Position: U0 U1 U2 U3 U4 U5 U6 U7 U8 | R0-R8 | F0-F8 | D0-D8 | L0-L8 | B0-B8

Example: W W W W W W W W W | R R R... | F F F... | ...

W=White, Y=Yellow, R=Red, O=Orange, G=Green, B=Blue

Internal Cube Representation

- **cp** (cornerPositions): 8-element array [0-7]
- **co** (cornerOrientations): 8-element array [0-2]
- **ep** (edgePositions): 12-element array [0-11]
- **eo** (edgeOrientations): 12-element array [0-1]

Learning Resources

Cube Theory

- **Kociemba Algorithm** - two-phase approach: layer solving + final layer
- **Cube State Space** - $\sim 4.3 \times 10^{19}$ valid configurations
- **Move Set** - 18 basic moves (6 faces $\times$ 3 rotations each)

Related Standards

- **WCA (World Cube Association)** - official notation standard
- **Singmaster Notation** - R, U, F, B, L, D moves + modifiers (' , 2)

Future Enhancement Ideas

- ☐ Mobile app version (React Native)
- ☐ Multiplayer solve racing
- ☐ Advanced solver strategies (Roux, CFOP, Petrus)
- ☐ Cube pattern recognition (e.g., PB detection)
- ☐ Solution optimization (fewer move rotations)
- ☐ Tutorial/beginner solving guides
- ☐ Cube scramble generator

- ☐ Statistics/timing tracking
- ☐ Dark mode toggle
- ☐ Multi-language support

License & Attribution

Original Developer: GitarNav Goel

Solver Credit: CubeJS library (Kociemba implementation)

Styling Credit: Modern UI design principles, professional typography

Support & Troubleshooting

Common Issues

Issue	Solution
"Camera not found"	Grant browser camera permission, check if device has camera
"Solver initialization fails"	Refresh page, clear browser cache, check internet connection
"Invalid cube" message	Verify colors match physical cube exactly, check Verify screen
"3D cube doesn't display"	Update browser, enable WebGL, check GPU acceleration
"Slow animation"	Reduce playback speed, close other tabs, check CPU usage

Debug Mode

Open browser console and run:

```
app.cubeState.faces // View current cube state
app.currentPhase // View current phase (1-4)
app.solver.initialized // Check if solver ready
```

Version: 1.0.0

Last Updated: July 2026

Repository: [GitHub - Rubix-Cube-Solver](#)